Code Explanation

Unit Test Explanation for main Module

The provided code is a unit test suite written in Python using the Pytest framework. It tests the `run_pipeline` function from the `src.main` module, covering both successful execution and error handling scenarios.

Overview of the Code
The test suite consists of two test functions: `test_run_pipeline` and `test_run_pipeline_exception`. Both tests utilize mocking to isolate dependencies and focus on the logic within the `run_pipeline` function. The tests verify that the pipeline steps are executed in the correct order and that error handling is properly implemented.

Step 1: Importing Necessary Modules and Defining Test Functions
This step involves importing the required modules, including Pytest and the `unittest.mock` module for mocking dependencies. The test functions `test_run_pipeline` and `test_run_pipeline_exception` are defined to test the `run_pipeline` function under different scenarios.

```
import pytest
from unittest.mock import patch, MagicMock
from src.main import run_pipeline

def test_run_pipeline():
    """Test the full pipeline execution."""

def test_run_pipeline_exception():
    """Test pipeline error handling."""
```

Step 2: Mocking Dependencies for Successful Pipeline Execution
In this step, the `test_run_pipeline` function uses the `@patch` decorator to mock several dependencies of the `run_pipeline` function. These include `get_spark_session`, `extract_source_data`, `transform_policy_data`, `load_to_target`, and `logging`. The mocks are configured to return specific values or behave in certain ways.

```
with patch('src.main.get_spark_session') as mock_get_spark,
     patch('src.main.extract_source_data') as mock_extract,
     patch('src.main.transform_policy_data') as mock_transform,
     patch('src.main.load_to_target') as mock_load,
     patch('src.main.logging') as mock_logging:

    # Configure mocks
    mock_spark = MagicMock()
    mock_get_spark.return_value = mock_spark

    mock_source_tables = MagicMock()
    mock_extract.return_value = mock_source_tables

    mock_transformed_data = MagicMock()
    mock_transform.return_value = mock_transformed_data
```

Step 3: Calling the `run_pipeline` Function and Verifying Pipeline Steps
The `run_pipeline` function is called within the context of the mocked dependencies. The test then verifies that each pipeline step was executed in the correct order by asserting that the corresponding mocks were called as expected.

```
# Call the function
run_pipeline()

# Verify the pipeline steps were executed in order
mock_get_spark.assert_called_once()
mock_extract.assert_called_once_with(mock_spark)
mock_transform.assert_called_once_with(mock_source_tables)
mock_load.assert_called_once_with(mock_spark, mock_transformed_data)
mock_spark.stop.assert_called_once()
```

Step 4: Testing Error Handling in the `run_pipeline` Function
The `test_run_pipeline_exception` function tests how the `run_pipeline` function handles exceptions. It mocks the `get_spark_session`, `extract_source_data`, and `logging` dependencies. The `extract_source_data` mock is configured to raise an exception.

```
with patch('src.main.get_spark_session') as mock_get_spark,
     patch('src.main.extract_source_data') as mock_extract,
     patch('src.main.logging') as mock_logging:

    # Configure mock to raise an exception
    mock_extract.side_effect = Exception("Test error")
    mock_spark = MagicMock()
    mock_get_spark.return_value = mock_spark

    # Call the function and expect it to raise the exception
    with pytest.raises(Exception, match="Test error"):
        run_pipeline()

    # Verify spark session was still stopped
    mock_spark.stop.assert_called_once()
```